TEXT ANALYTICS

Feature Selection, Reduction and Importance

Francesco Spinnato

* francesco.spinnato@unipi.it
University of Pisa



Which Words Matter?

A Question We Keep Asking

We have **ranked words** many times, under different names: frequency, TF-IDF, PMI, top terms of a cluster, weights of a classifier.

Each time, "important" meant something different. The useful question is always: **important for what?**

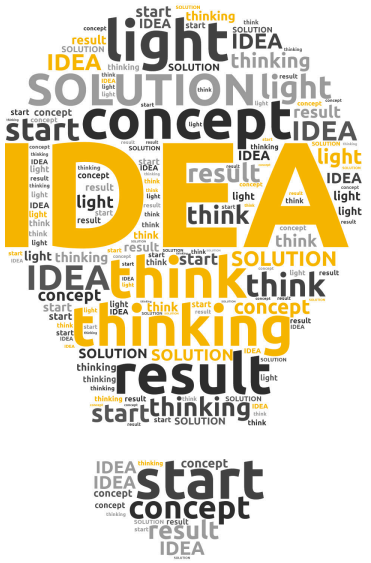
- for the **corpus**: how common is the word, which words go together?
- for a **document**: which words characterize this text?
- for a **group of documents**: which words describe a cluster or a topic?
- for a **class**: which words tell the classes apart?
- for a **model**: which words does the classifier actually rely on?

Important for the Corpus

Frequency. The simplest (but misleading) score. By Zipf's law the top of the ranking is made of function words (*the*, *of*, *and*): frequent, but they say nothing about the content. Removing **stopwords** was our first way of saying that frequent does not mean informative.

Association. For collocations, we scored **pairs** of words instead: how much more often do w_1 and w_2 occur together than chance predicts?

- Both start from the same **null hypothesis**: the two words are independent.
- **PMI/PPMI** measure how far the observed co-occurrence is from that, but favor rare words.
- **Statistical tests** (χ^2, t-test) ask whether the departure can be explained by chance.



Important for a Document

TF-IDF scores a word **in a document**, relative to the corpus:

$$\text{TF-IDF}(w, d) = \text{TF}(w, d) \times \log \frac{N}{\text{DF}(w)}$$

- **TF** rewards words the document repeats;
- **IDF** penalizes words that occur in many documents, so stopwords score close to zero without a list.

Sorting the words of a document by TF-IDF gives a rough list of its **keywords**. The importance is **local**: the same word can be the top term of one document and irrelevant in another.

Important for a Group of Documents

Unsupervised methods describe what they find through a ranking of words:

- **A term as a topic**: score candidate terms with TF-IDF, then re-rank them with **MMR** (next slide), so that the chosen terms are both high-scoring and different from each other.
- **Clusters**: a k-means centroid $\boldsymbol{\mu}_j$ is the average TF-IDF vector of its documents, so its largest coordinates are the words that describe the cluster.
- **Topics**: a topic θ_j is a distribution over words; we read it through its most probable words, those with the largest $P(w \mid \theta_j)$.

In all three cases, nobody told the method what the groups were: the words explain the groups **after** they have been found.

Re-ranking with Maximal Marginal Relevance

Picking the top k terms by score often returns **important but similar** words. **MMR** picks one term at a time, trading the score off against the similarity to the terms already chosen.

Ranked by TF-IDF score

1	2	3	4	5	6
sport	***football***	***soccer***	*travel*	*match*	*politics*

Re-ranked with MMR

1	2	3	4	5	6
sport	***travel***	***politics***	*football*	*match*	*soccer*

Important for a Class

With labels, a word is important if it is much more likely in one class than in the other. The **log ratio** of its (add-one smoothed) relative frequencies in the two classes measures this, one word at a time, with **no model trained**:

$$\ln \frac{\hat{P}(w \mid +)}{\hat{P}(w \mid -)} \qquad \textit{predictable} : \ln \frac{1/29}{2/34} = -0.53 \quad \textit{no} : -0.53 \quad \textit{fun} : \ln \frac{2/29}{1/34} = +0.85$$

Like PMI, the ratio is extreme for rare words; smoothing, or a stronger prior (Monroe et al., 2008), tames them. χ^2 and information gain, later in this lecture, answer the same question.

As with the scores of the previous slides, this is a **statistic of the data**: nothing has been fitted.

Important for a Model

Trained models carry their own importance scores:

- **Naive Bayes**: in log space its score is the log prior ratio plus one log ratio per word, exactly the ratios of the previous slide. For *"predictable with no fun"*: $-0.41 - 0.53 - 0.53 + 0.85 = -0.62 < 0 \Rightarrow$ negative.
- **Logistic regression** (and linear SVMs): one weight w_j per word, the change in log-odds for one more unit of x_j. Sort the weights to get the words pushing towards each class; with L1 regularization most weights are exactly zero.
- **Decision trees**: the word at the **root** is the one with the highest information gain; the deeper a word is used, the smaller the part of the data it decides on.

These scores describe **the model**, not the data: two models trained on the same corpus can rely on different words.

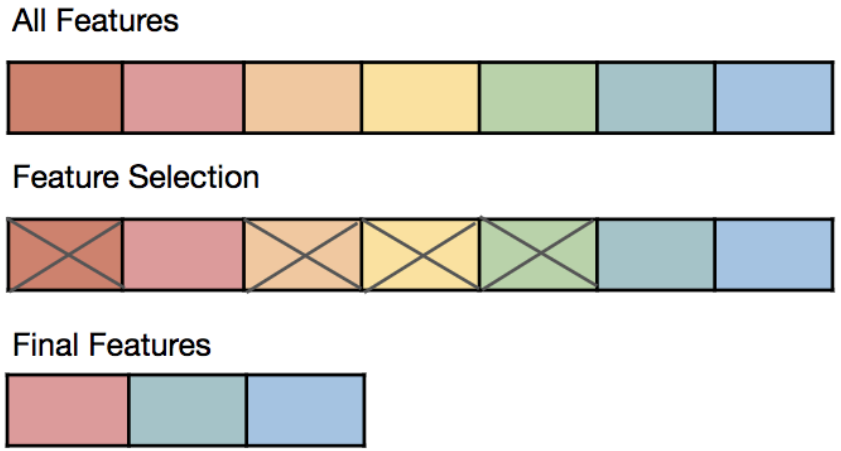
Feature Selection for Text Analytics

Why Select Features?

With bag-of-words features, every word type is a feature: $M = |V|$, easily tens of thousands (Herdan's law). Most of them are **rare** (Zipf), and most are **irrelevant** to any given task.

Feature selection keeps a subset of the original features and removes those unlikely to generalize. It pays off in:

- **generalization**: fewer spurious features to overfit
- **cost**: smaller vectors, faster training and prediction
- **interpretability**: a model over 500 words can be read, one over 50,000 cannot



Three Families

Family	How it works	Examples	Cost
Filter	score every feature on its own, usually against the labels, before training; keep the top k	document frequency, information gain, χ^2, PMI	cheap
Wrapper	search for the subset of features on which a model performs best	forward selection, backward elimination	expensive: one model per subset tried
Embedded	selection happens during training	L1 regularization, decision trees	free with the model

With text, **filters** are by far the most common: they scale to tens of thousands of features, and they are independent of the classifier.

Filter Scores for Text

Each score answers a slightly different question about a word w and class c:

- **Document frequency**: in how many documents does w occur? Drop the rarest words, and possibly the most common ones: the stopword and rare word removal seen in text preprocessing is already a filter.
- **Information gain**: how many bits of information does the presence of w give about the class? The same quantity decision trees use to choose a split.
- χ^2: are the occurrences of w and membership in c independent? The test we used for collocations, applied to a word and a class instead of to two words.
- **PMI**, again from collocations: how much more often do w and c co-occur than independence predicts? As there, it favors rare words.

Information Gain

A split pays off when the children are **purer** than the parent. Purity is measured by the **entropy** of the class proportions of a set of documents S, where S_c are the documents of S in class c:

$$H(S) = -\sum_{c \in Y} \frac{|S_c|}{|S|} \log_2 \frac{|S_c|}{|S|}$$

The **information gain** of splitting on a word w is the drop in entropy, where S_w and $S_{\neg w}$ are the documents of S that contain w and those that do not:

$$IG(S, w) = H(S) - \frac{|S_w|}{|S|} H(S_w) - \frac{|S_{\neg w}|}{|S|} H(S_{\neg w})$$

It reads as: how many **bits** of information about the class do we gain by knowing whether w occurs?

Information Gain: Worked Example

Ten emails, 5 spam and 5 ham, so $H(S) = 1$. Which word is the most informative?

Word w	S_w: present (spam / ham)	$S_{\neg w}$: absent (spam / ham)	$IG(S, w)$
meeting	0 / 3	5 / 2	$1 - (0.3 \times 0 + 0.7 \times 0.863) = \mathbf{0.396}$
free	4 / 1	1 / 4	$1 - (0.5 \times 0.722 + 0.5 \times 0.722) = 0.278$
the	5 / 4	0 / 1	0.108

- *meeting* wins, although it only occurs in 3 emails: its *present* branch is already **pure**.
- *free* is a strong spam word, but one ham email asks *"are you free for a meeting?"*
- *the* occurs almost everywhere and tells us almost nothing.

Chi-Square: A Word and a Class

Arrange the training documents in a 2×2 table of **observed** counts O: does the word occur, and is the document in the class?

Compare them with the counts E that **independence** would produce:

$$E_{ij} = \frac{(\text{row } i \text{ total}) \times (\text{column } j \text{ total})}{N} \qquad \chi^2(w, c) = \sum_{i \in \{w, \neg w\}} \sum_{j \in \{c, \neg c\}} \frac{(O_{ij} - E_{ij})^2}{E_{ij}}$$

The further the observed counts are from what independence predicts, the larger χ^2, and the more the word tells us about the class.

Chi-Square: Worked Example

The same ten emails as before, for the word *free* and the class spam:

	spam	ham	total
free present	4 *(E = 2.5)*	1 *(E = 2.5)*	5
free absent	1 *(E = 2.5)*	4 *(E = 2.5)*	5

$$\chi^2 = 4 \times \frac{(1.5)^2}{2.5} = 3.6 \qquad \chi^2(meeting) = 4.29 \qquad \chi^2(the) = 1.11$$

Same ranking as information gain: *meeting*, *free*, *the*.

Counts this small are for illustration only: the χ^2 approximation needs expected counts of about 5 or more.

Feature Selection in Practice

- **Score, rank, keep the top k.** How many to keep is a hyperparameter: choose k on validation data, never on the test set.
- **Score on the training set only.** Supervised filters use the labels. Computing them on all the data, test set included, lets test labels leak into the model: the evaluation will look better than reality.
- **Per class or overall.** Scores are computed for a word and a class; with many classes, take the maximum or the average over classes.
- **Embedded alternatives.** L1-regularized logistic regression sets most weights to exactly zero, selecting features while it trains.

Dimensionality Reduction

From Selecting to Building Features

The document-term matrix $\mathbf{X} \in \mathbb{R}^{N \times M}$ is large, sparse, and redundant: *car* and *automobile* are two separate columns, although they carry almost the same information.

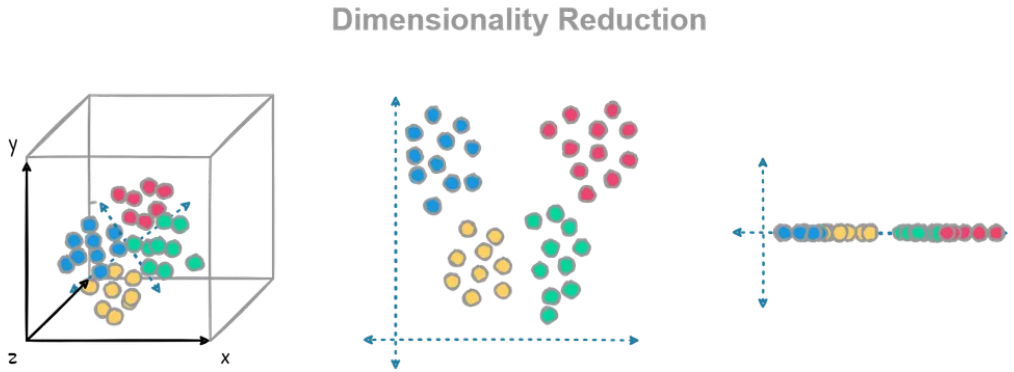
Feature selection **keeps** original features, **dimensionality reduction** builds $k \ll M$ **new** features, each a combination of the original ones, and represents every document as a dense vector $\tilde{\mathbf{x}}_d \in \mathbb{R}^k$.

From Selecting to Building Features

Dimensionality reduction is used for two different purposes:

- **as features**: k in the tens or hundreds, as input to a classifier or to clustering
- **for visualization**: $k = 2$ or 3, to look at a corpus (or at embeddings) on a plot

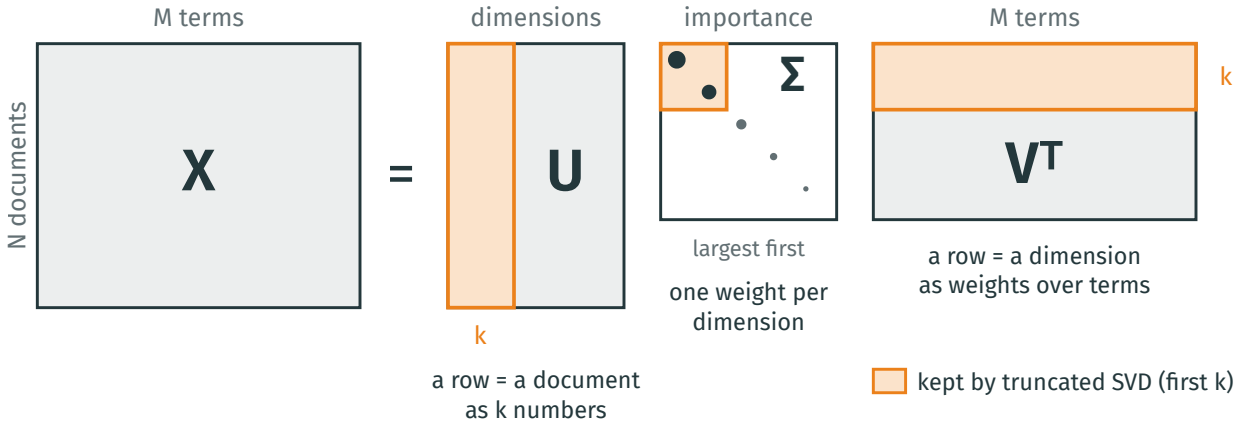
The price: unlike selected features, the new dimensions are no longer words, so they are harder to read.



Singular Value Decomposition (SVD)

SVD splits the document-term matrix $\mathbf{X}$ into the product of three matrices, $\mathbf{X} = \mathbf{U}\,\mathbf{\Sigma}\,\mathbf{V}^\top$:

- $\mathbf{U}$: every **document** described by hidden **dimensions**, each a group of words that tend to occur together
- $\mathbf{\Sigma}$: how **important** each dimension is, from the most to the least important
- $\mathbf{V}^\top$: how much every **term** contributes to each dimension



Multiplying the three gives back $\mathbf{X}$ exactly, so nothing is lost or compressed yet. What changes is that the dimensions are now **sorted by importance**.

The bold $\mathbf{V}$ is a matrix, not the vocabulary V.

Truncated SVD and Latent Semantic Analysis

Keep only the first k dimensions, the orange parts of the picture, and drop the rest: $\mathbf{X} \approx \mathbf{U}_k \, \mathbf{\Sigma}_k \, \mathbf{V}_k^\top$. Every document is now described by k numbers instead of M.

- Little is lost: the dropped dimensions are the least important ones.
- Words that occur in similar documents end up on the same dimensions, so *car* and *automobile* get close, even if no document uses both.
- It works directly on the **sparse** matrix, so it scales to real vocabularies.
- Each row of $\mathbf{V}_k^\top$ is a vector over the vocabulary, like a k-means centroid: its largest weights are the words that describe that dimension.

Applied to a TF-IDF document-term matrix, it is called **Latent Semantic Analysis** (LSA, Deerwester et al., 1990). Typical k: 100 to 300.

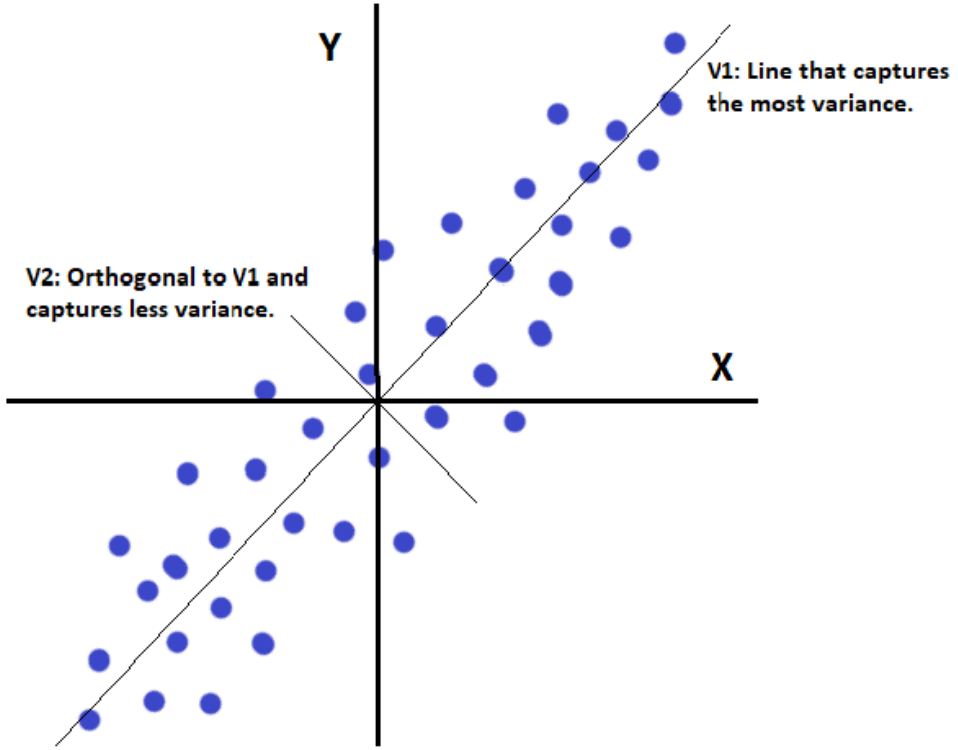
The latent dimensions can have negative weights, so they are hard to read as topics: this is what probabilistic topic models fix.

Principal Component Analysis (PCA)

PCA finds orthogonal directions, the **principal components**, ordered by how much **variance** of the data they capture.

It is the SVD of the **centered** matrix, where the mean of every column is subtracted first.

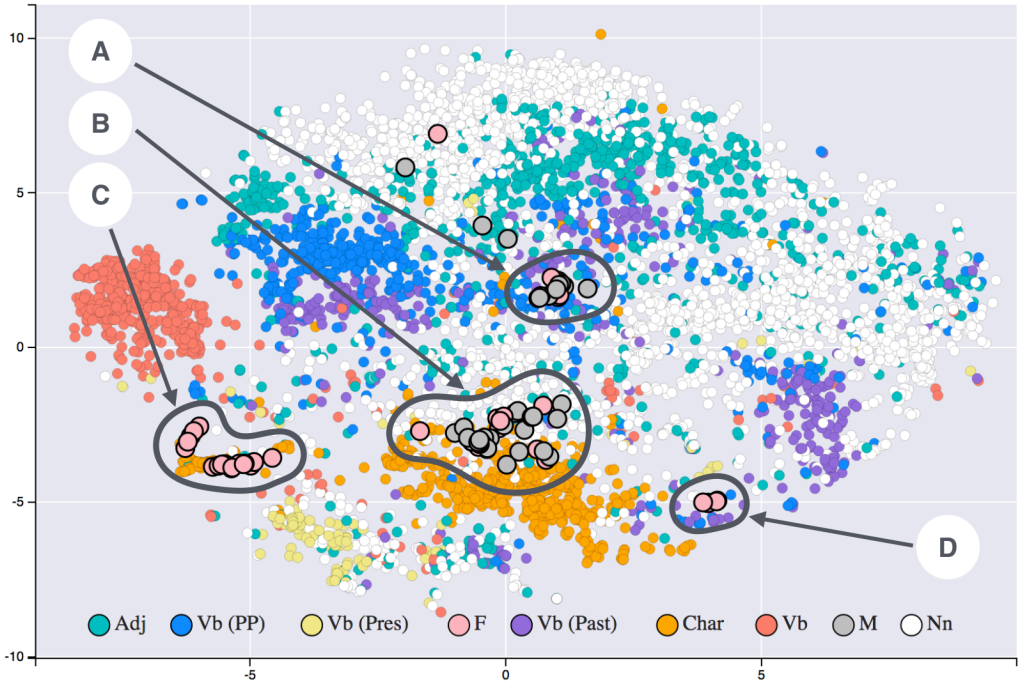
- Linear, fast, deterministic.
- For a first plot of a corpus: keep 2 components.
- On text, centering turns the zeros into non-zeros and **destroys sparsity**: on a document-term matrix, truncated SVD is used instead.



t-SNE

t-SNE (van der Maaten & Hinton, 2008) is **nonlinear** and preserves **local neighborhoods**: points that are close in $\mathbb{R}^M$ should stay close in $\mathbb{R}^2$.

- It turns distances into "probability of being neighbors", in the original space and in 2D, and moves the 2D points until the two match.
- **Perplexity** sets roughly how many neighbors each point considers (typically 5 to 50).

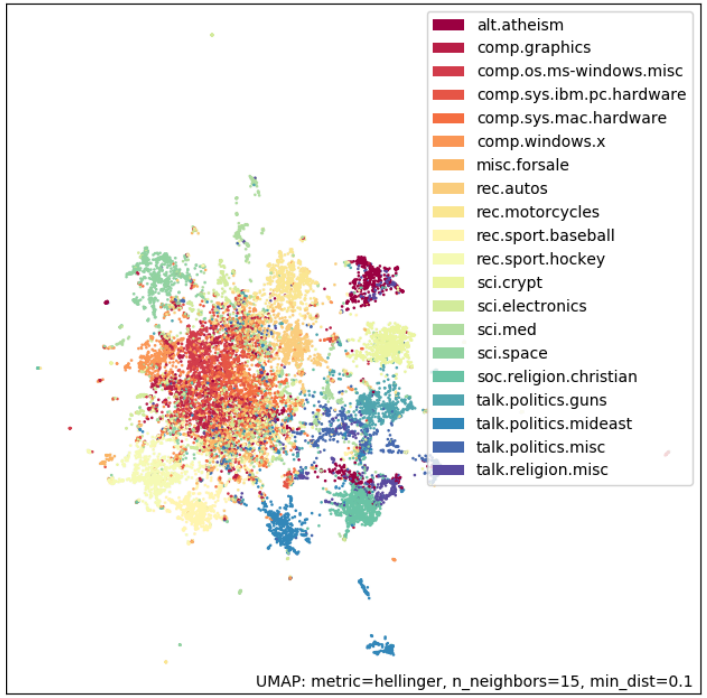


Read the plots with care: cluster **sizes** and **distances between clusters** mean little, and different runs or perplexities give different pictures. It is a **visualization** tool only: it cannot place new documents on an existing map.

UMAP

UMAP (McInnes, Healy & Melville, 2018) follows the same idea as t-SNE: build a graph of the nearest neighbors of every point, then find a 2D (or k-dimensional) layout with the same neighbors. Compared to t-SNE:

- much **faster**, so it scales to hundreds of thousands of documents
- tends to keep more of the **global** structure (how clusters are arranged)
- can **project new documents** onto an existing map, so it is also used as a feature step, e.g., before clustering document embeddings
- main parameters: `n_neighbors` (local vs global view) and `min_dist` (how tightly points are packed)



The same t-SNE distance warnings apply.

Dimensionality Reduction: Summary

Method	Linear	Preserves	Typical use	New documents
SVD	yes	everything (exact)	the building block	yes
Truncated SVD / LSA	yes	best rank-k approximation	features from TF-IDF	yes
PCA	yes	directions of max variance	first plot, dense data	yes
t-SNE	no	local neighborhoods	2D visualization	no
UMAP	no	local, partly global structure	visualization, before clustering	yes

In practice, a common pipeline for plotting a corpus is **TF-IDF → truncated SVD to ~50 dimensions → t-SNE or UMAP to 2.**

Interpreting a Model

Why Interpret a Text Classifier?

Accuracy says **how often** a model is right, not **why**. Looking at the features it relies on helps to:

- **debug**: find features that should not matter, such as leaked metadata or artifacts of how the data was collected
- **trust**: check that the decisions rest on the content, not on shortcuts
- **learn about the task**: which words signal a complaint, a fake review, a topic

Two kinds of explanation:

- **global**: which features matter for the model **overall**
- **local**: why the model made **this** prediction for **this** document

A Cautionary Tale

Classify posts of the **20 Newsgroups** corpus as *Christianity* or *Atheism*. An SVM on unigrams reaches **94%** accuracy on held-out data.

Its explanations, however, rest on words such as ***Posting***, ***Host*** and ***Re***: words from the **email headers**, unrelated to religion. *Posting* occurs in 22% of the training posts, and 99% of those are *Atheism*.

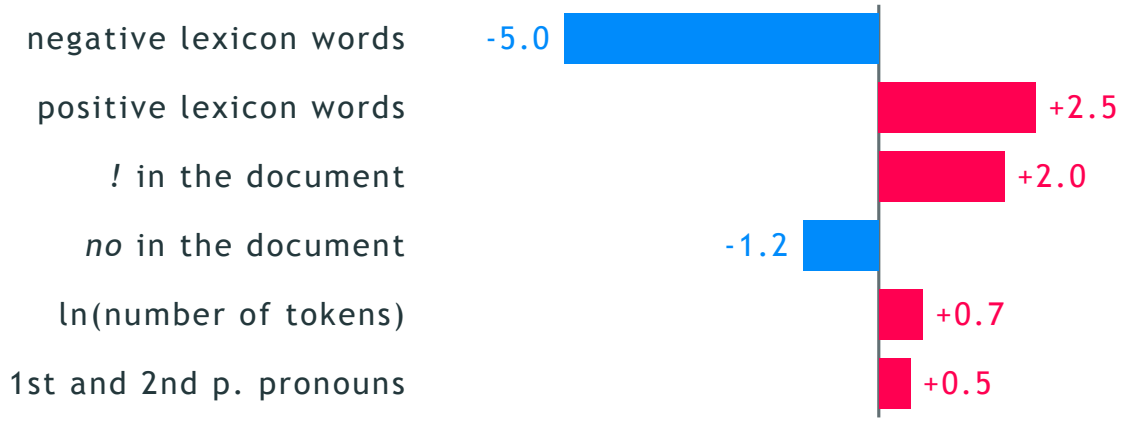
- The test set came from the same collection, so the shortcut worked there too, and the accuracy looked excellent.
- On posts from anywhere else, the classifier would not generalize.
- Even with the headers removed, it picks up the **names** of prolific posters of the two newsgroups: another shortcut.

Only reading the important features revealed the problem.

A Global Explanation for a Linear Model

The **weights** of the logistic regression example, sorted by size: the **global** importance of each feature, the same for every document.

- A negative lexicon word costs 5.0 in log-odds, **twice** what a positive one earns.
- *!* has a large weight, length a small one.



The weights alone do not explain a **single** prediction: a feature with a large weight may be absent from the document (*!*), and one with a small weight may have a large value (length, $\ln 66 = 4.19$).

A Local Explanation for a Linear Model

The weights hold for every document. For **one** prediction, look at each feature's **contribution** $w_j x_j$: the score is a **sum**, so the contributions add up to it. Here are the words that fire each feature in the review:

It's hokey. There are virtually no surprises, and the writing is second-rate. So why was it so enjoyable? For one thing, the cast is great. Another nice touch is the music. I was overcome with the urge to get off the couch and start dancing. It sucked me in, and it'll do the same to you.

- red: pushes towards **positive** (positive lexicon words, pronouns)
- blue: pushes towards **negative** (negative lexicon words, *no*)

This is what explanation tools draw on top of a text: each word colored by how much, and in which direction, it moved the prediction.

A Local Explanation for a Linear Model

$$\underbrace{2.5 \times 3}_{\text{pos. lexicon}} + \underbrace{(-5.0) \times 2}_{\text{neg. lexicon}} + \underbrace{(-1.2) \times 1}_{no} + \underbrace{0.5 \times 3}_{\text{pronouns}} + \underbrace{2.0 \times 0}_{!} + \underbrace{0.7 \times 4.19}_{\ln(\text{tokens})} + \underbrace{0.1}_{b}$$

$$= +7.50 - 10.00 - 1.20 + 1.50 + 0 + 2.93 + 0.1 = 0.83 \qquad \sigma(0.83) = 0.70$$

- The two negative lexicon words alone (-10.00) outweigh the three positive ones ($+7.50$); the review ends up positive thanks to the pronouns and the **length** feature.
- The explanation is **exact**, no approximation involved: the contributions add up to the score.
- The log-likelihood ratios of Naive Bayes are read in the same way.

Reading Weights with Care

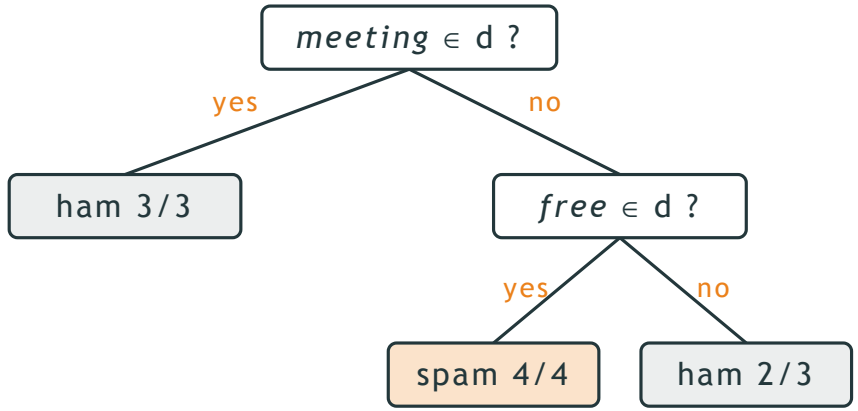
The weights of a linear model are a global explanation, but:

- **Scale.** A weight is per unit of x_j: weights of features on different scales (counts, TF-IDF, length) cannot be compared directly. Standardize, or compare words only with words.
- **Correlated features.** *excellent* and *superb* can share the credit, so both look less important than they are; with L1, one may be kept and the other set to zero, almost at random.
- **Regularization.** Changing its strength changes the ranking, especially among the less important words.

Importance in Trees and Forests

A tree has **no weight per word**: a word matters by **where** it is used and **how much** its splits purify the data.

- **Local**: the path of a document is its explanation, a rule like *IF* meeting $\notin d$ *AND* free $\in d$ *THEN* spam.
- **Global**: the **impurity importance** of a word sums the information gain of every split on it, weighted by the share of documents reaching that node. Forests average it over their trees.



A word is globally important if it makes **clean** splits for **many** documents. *meeting* sits at the root and sorts all 10 emails; *free* only sorts the 7 that reach it, so it counts less.

Permutation Importance

A **model-agnostic** global score: how much does the model need feature j?

1. Compute the score of the trained model on the **validation** set.
2. **Shuffle** column j of the validation set, breaking its link with the labels, and compute the score again.
3. The **drop** in score is the importance of feature j. Repeat with several shuffles and average.

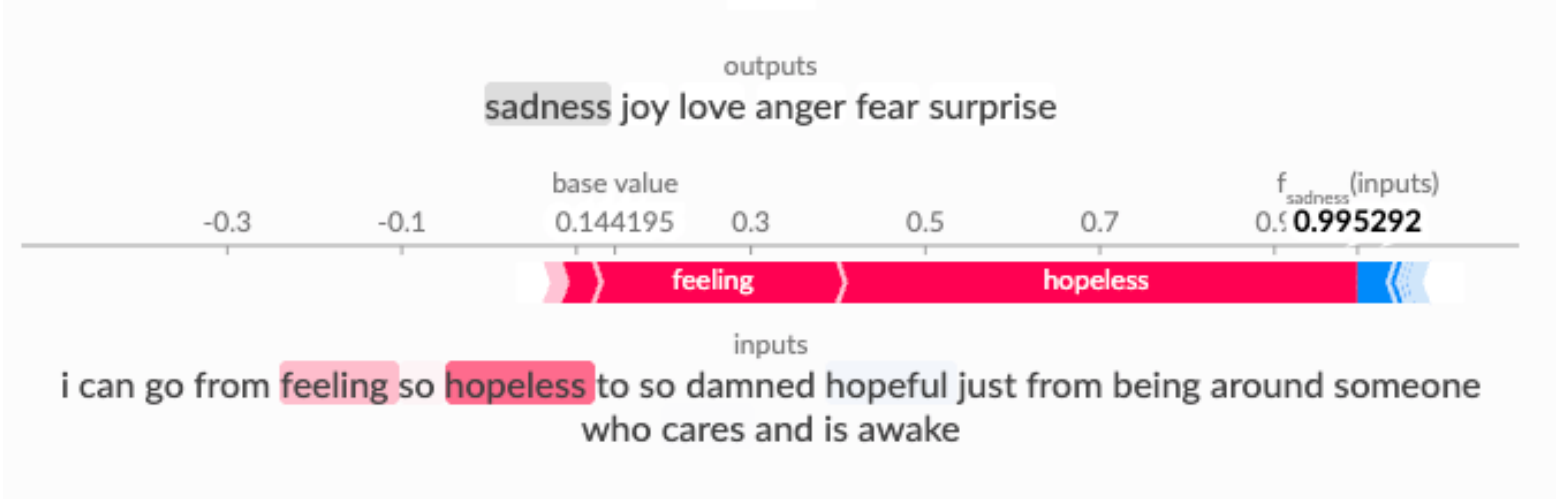
Pros and Cons:

- It works for **any** model, and is measured on held-out data, not on the training data.
- With tens of thousands of words it is expensive: apply it to the top features, or to **groups** of features (all header tokens, all punctuation features, all metadata).
- Correlated features hide each other: if *excellent* is shuffled, *superb* still carries the information, and neither seems important.

Beyond: Explaining Any Model

For models that are not linear, such as neural networks later in the course, there is no $w_j x_j$ to read. **Local, model-agnostic** methods approximate it:

- **LIME** (Ribeiro et al., 2016): remove words from the document at random, see how the prediction changes, and fit a small linear model that imitates the classifier **around that document**.
- **SHAP** (Lundberg & Lee, 2017): share the prediction among the features in the only way that satisfies a few properties (Shapley values, from game theory).



Sources: Textbooks

- Molnar, *Interpretable Machine Learning*, ch. 6, 7, 9, 17, 18, 23
- Manning, Raghavan & Schütze (2008), *Introduction to Information Retrieval*, ch. 13.5, 18
- James, Witten, Hastie & Tibshirani, *An Introduction to Statistical Learning*, 2nd ed., ch. 12.2

Sources: Papers

- Yang & Pedersen (1997), *A Comparative Study on Feature Selection in Text Categorization*, ICML
- Monroe, Colaresi & Quinn (2008), *Fightin' Words: Lexical Feature Selection and Evaluation for Identifying the Content of Political Conflict*, Political Analysis 16(4)
- Carbonell & Goldstein (1998), *The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries*, SIGIR
- Deerwester, Dumais, Furnas, Landauer & Harshman (1990), *Indexing by Latent Semantic Analysis*, JASIS 41(6)
- van der Maaten & Hinton (2008), *Visualizing Data using t-SNE*, JMLR 9
- McInnes, Healy & Melville (2018), *UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction*, arXiv:1802.03426
- Breiman (2001), *Random Forests*, Machine Learning 45
- Ribeiro, Singh & Guestrin (2016), *"Why Should I Trust You?": Explaining the Predictions of Any Classifier*, KDD
- Lundberg & Lee (2017), *A Unified Approach to Interpreting Model Predictions*, NeurIPS